

Informe de Desarrollo: TP2 - Back-end para servir el sitio web

Asignatura: Aplicaciones Web II

Institución: IES21

Profesor: Andrés Senn

Integrantes: Luján Federico, Giménez Fabrizio

1. Introducción

El presente informe detalla los avances técnicos realizados en el marco del desarrollo de la plataforma web para la rotisería "Como en casa". En esta etapa, el equipo ha tomado la decisión estratégica de iniciar el desarrollo desde cero, prescindiendo de los proyectos realizados en la materia anterior (AW1). Esta decisión nos permite implementar una arquitectura moderna, limpia y escalable, alineada estrictamente con los requerimientos actuales de desarrollo back-end exigidos en esta cátedra.

El proyecto está concebido no solo como un catálogo de productos para el público, sino como un sistema integral de gestión. La visión final contempla una plataforma operativa bajo una modalidad dual:

- **Interfaz para el Cliente:** Donde el usuario podrá navegar por el menú, seleccionar productos y gestionar su pedido de forma intuitiva.
- **Interfaz para el Personal:** Un módulo de gestión interna que permitirá a los empleados registrar ventas en tiempo real, controlar el stock y administrar los pedidos entrantes de manera eficiente.

Cabe destacar que, si bien la versión actual se centra en la interfaz de catálogo y en una experiencia de usuario básica, la arquitectura del sistema ha sido diseñada pensando en esta integridad funcional. Somos conscientes de que, al ser una implementación nueva, aún restan integrar diversas funcionalidades críticas y características visuales de la planificación original. No obstante, el desarrollo se ha planteado de forma modular, lo que garantiza que en las entregas sucesivas se irán incorporando estas capacidades de manera orgánica, aprovechando la flexibilidad que nos brinda el entorno de Node.js, hasta alcanzar una herramienta totalmente operativa y profesional.

2. Alcance del Proyecto y Visión Evolutiva

Al tratarse de una implementación nueva, el sitio se encuentra en una etapa inicial de desarrollo. Actualmente, el foco ha sido establecer la infraestructura base (servidor y conexión a datos).

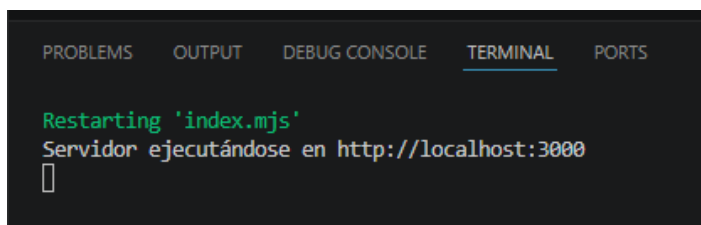
- **Funcionalidades actuales:** Catálogo dinámico, estructura visual básica y sistema de carrito inicial.
- **Hoja de ruta (Roadmap):** Se deja constancia de que las funcionalidades y características prometidas en la planificación original se irán integrando de forma modular en los próximos Trabajos Prácticos, hasta alcanzar la versión final totalmente funcional.

3. Tecnologías Utilizadas

- **Servidor:** Node.js y Express JS para el despacho de archivos y gestión de rutas.
- **Base de Datos:** Consumo de API externa a través de MockAPI, garantizando que la información de los productos no sea estática.
- **Front-end:** HTML5 semántico, CSS3 (fuentes personalizadas vía Google Fonts) y JavaScript asíncrono.
- **Entorno:** Desarrollo sobre sistemas Linux (WSL) y gestión de paquetes con NPM.

4. Detalles de la Implementación

- **Arquitectura de Servidor:** Se configuró un punto de entrada (`app.js`) que inicializa el servidor Express. Se definió una carpeta pública (`/recursos`) para servir el contenido estático de manera segura.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Restarting 'index.mjs'
Servidor ejecutándose en http://localhost:3000
█
```



```
JS index.mjs > ...
1  import express from 'express';
2  import path from 'path';
3  import { fileURLToPath } from 'url';
4
5  const app = express();
6  const PORT = 3000;
```

- **Gestión de Datos:** Se implementa la lógica de `fetch` para conectar el front-end con el endpoint de MockAPI. Esto asegura que el sistema de carga de productos cumpla con el requisito de "web totalmente funcional" en cuanto a la obtención de datos externos

FETCH:

```

const contenedorProductos = document.getElementById("productos");
let carrito = []; // Lista para guardar los productos seleccionados

fetch("https://69c566e48a5b6e2dec2c6298.mockapi.io/api/v1/comidas")
  .then(response => response.json())
  .then(data => {
    data.forEach(producto => {
      contenedorProductos.innerHTML += `
        <div class="card">
          
          <div class="contenido-card">
            <h2>${producto.nombre}</h2>
            <p>${producto.descripcion}</p>
            <span class="precio">${producto.precio}</span>

            <!-- Agregamos el botón aquí -->
            <button class="btn-agregar" onclick="agregarAlCarrito('${producto.nombre}', ${producto.precio})">
              Agregar al carrito
            </button>
          </div>
        </div>
      `;
    });
  })
  .catch(error => console.log(error));

```

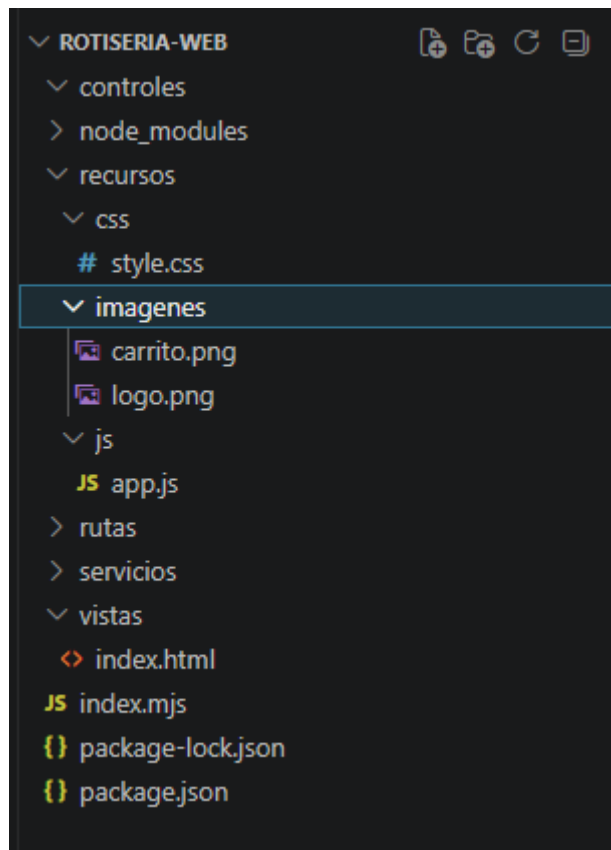
MOCKAPI:

```

[
  {
    "nombre": "Pizza Muzzarella",
    "precio": 8500,
    "descripcion": "Pizza clásica con abundante queso",
    "imagen": "https://images.unsplash.com/photo-1565299624946-b28f40a0ae38"
  },
  {
    "nombre": "Hamburguesa Completa",
    "precio": 6200,
    "descripcion": "Hamburguesa doble con papas fritas",
    "imagen": "https://images.unsplash.com/photo-1568901346375-23c9450c58cd"
  },
  {
    "nombre": "Pancho Especial",
    "precio": 3500,
    "descripcion": "Pancho con cheddar y panceta",
    "imagen": "https://images.unsplash.com/photo-1619740455993-9e612b1af08a"
  },
  {
    "nombre": "Papas con Cheddar",
    "precio": 4800,
    "descripcion": "Papas fritas con cheddar y verdeo",
    "imagen": "https://images.unsplash.com/photo-1576107232684-1279f390859f"
  },
  {
    "nombre": "Tacos Mexicanos",
    "precio": 7000,
    "descripcion": "Tacos con carne y salsa picante",
    "imagen": "https://images.unsplash.com/photo-1552332386-f8dd00dc2f85"
  },
  {
    "nombre": "Pollo al Spiedo",
    "precio": 12000,
    "descripcion": "Pollo asado con papas",
    "imagen": "https://images.unsplash.com/photo-1600891964092-4316c288032e"
  },
  {
    "nombre": "Lomito Completo",
    "precio": 8900,
    "descripcion": "Lomito con jamón, queso y huevo",
    "imagen": "https://images.unsplash.com/photo-1550317138-10000687a72b"
  },
  {
    "nombre": "Shawarma",
    "precio": 7600,
    "descripcion": "Wrap árabe con carne y vegetales",
    "imagen": "https://images.unsplash.com/photo-1529006557810-274b9b2fc783"
  },
  {
    "nombre": "Pizza Pepperoni",
    "precio": 9500,
    "descripcion": "Pizza con pepperoni extra",
    "imagen": "https://images.unsplash.com/photo-1513104890138-7c749659a591"
  },
  {
    "nombre": "Burger BBQ",
    "precio": 7300,
    "descripcion": "Hamburguesa con salsa barbacoa",
    "imagen": "https://images.unsplash.com/photo-1565299624946-b28f40a0ae38"
  }
]

```

- **Organización del Código (Modularización):** El proyecto sigue una estructura de directorios profesional para facilitar el escalado:
 - `/recursos/css`: Estilos visuales.
 - `/recursos/js`: Lógica del lado del cliente.
 - `/recursos/imagenes`: Recursos gráficos locales.
 - Raíz: Configuración de servidor y dependencias (`package.json`).



5. Conclusión

Este TP sienta las bases operativas de la rotisería. Si bien el sitio es una versión simplificada respecto al objetivo final, cumple rigurosamente con los requisitos de entrega: un servidor Node.js operativo, consumo de APIs externas y una estructura de archivos organizada para el crecimiento del proyecto en las siguientes etapas de la materia.



ROTISERÍA COMO EN CASA

"TU COMIDA FAVORITA, LISTA CUANDO LA NECESITÁS."





Pizza Muzzarella

Pizza clásica con abundante queso

\$8500

Agregar al carrito



Hamburguesa Completa

Hamburguesa doble con papas fritas

\$6200

Agregar al carrito



Pancho Especial

Pancho con cheddar y panceta

\$3500

Agregar al carrito



Papas con Cheddar

Papas fritas con cheddar y verdeo

\$4800

Agregar al carrito



Tacos Mexicanos

Tacos con carne y salsa picante

\$7000

Agregar al carrito



Pollo al Spiedo

Pollo asado con papas

\$12000

Agregar al carrito



Lomito Completo

Lomito con jamón, queso y huevo

\$8900

Agregar al carrito



Shawarma

Wrap árabe con carne y vegetales

\$7600

Agregar al carrito



Pizza Pepperoni

Pizza con pepperoni extra

\$9500

Agregar al carrito



Burger BBQ

Hamburguesa con salsa barbaoca

\$7300

Agregar al carrito



Alitas Picantes

Alitas de pollo con salsa picante

\$6800

Agregar al carrito



Cheesecake

Cheesecake con frutos rojos

\$5400

Agregar al carrito

Como en casa

La mejor rotisería de la zona, llevando sabor a tu mesa desde siempre.

Contacto

↑ Calle Falsa 123, Córdoba
☎ +54 351 1234567

Horarios

Lunes a Sábados: 11:00 a 15:00 y 19:00 a 23:00
Domingos: Cerrado

© 2026 Rotisería Como en casa - Todos los derechos reservados

INFORME TÉCNICO: TRABAJO PRÁCTICO N° 3 – DESARROLLO DEL ABM COMPLETO

Institución: IES21

Materia: Aplicaciones Web II

Profesor: Andrés Senn

Integrantes: Luján Federico, Giménez Fabrizio

1. Introducción

El presente informe técnico expone el proceso de diseño, desarrollo e integración del módulo de administración completa (ABM/CRUD) para el sitio web de la rotisería **"Como en casa"**. Esta etapa representó el núcleo de la materia, ya que implicó la unificación definitiva de la interfaz de usuario (Frontend) con la lógica del servidor (Backend) y el motor de persistencia.

A lo largo del desarrollo, el equipo dedicó un tiempo considerable al análisis y comprensión de la arquitectura cliente-servidor. Comprender el flujo asíncrono de los datos, el ciclo de vida de una petición HTTP y la manipulación dinámica del DOM mediante JavaScript demandó un proceso de aprendizaje intensivo, lo que ralentizó inicialmente el avance pero garantizó una base sólida para lograr un sistema robusto, óptimo y libre de fallos estructurales.

2. Elección Tecnológica: PostgreSQL como Motor de Persistencia

Para este proyecto se optó de manera unánime por utilizar **PostgreSQL** como Sistema de Gestión de Bases de Datos Relacionales (RDBMS), descartando otras alternativas del mercado. Los fundamentos técnicos que justifican esta elección son:

- **Robustez y Cumplimiento ACID:** Al tratarse de un sistema de gestión de menús y pedidos, garantizar la atomicidad, consistencia, aislamiento y durabilidad de las transacciones es crítico para que los datos jamás se corrompan.
- **Manejo Eficiente de Tipos de Datos:** PostgreSQL ofrece un excelente soporte para tipos de datos numéricos exactos (**NUMERIC**), ideales para almacenar precios de productos sin sufrir los errores de redondeo que poseen los tipos flotantes en otros motores.
- **Escalabilidad y Concurrencia:** Mediante el uso de un pool de conexiones en Node.js (**pg.Pool**), el servidor puede atender múltiples peticiones concurrentes desde la interfaz administrativa de forma eficiente, reutilizando los canales de comunicación abiertos con la base de datos.

3. Arquitectura del Sistema y Flujo de Datos (La Lógica Comprendida)

Uno de los mayores desafíos del equipo fue asimilar cómo interactúan las capas del proyecto de forma asíncrona a través del protocolo HTTP. El flujo implementado se estructuró bajo el siguiente estándar lógico:

1. **El Frontend solicita o envía datos:** Mediante la API **fetch** de JavaScript, la interfaz administrativa realiza peticiones hacia endpoints específicos (ej. **/api/productos**).
2. **El Enrutador del Backend procesa la solicitud:** Express intercepta el método HTTP (**GET**, **POST**, **PUT** o **DELETE**) y el parámetro (ID) si corresponde.

3. **El Modelo ejecuta la consulta SQL:** A través del archivo `modelo.productos.mjs`, se despachan las sentencias SQL puras utilizando el cliente de PostgreSQL.
4. **La Base de Datos responde y el Frontend renderiza:** Los registros retornados por la base de datos se transforman en formato JSON en el servidor, viajan de vuelta por la red, y el Frontend se encarga de limpiar el contenedor HTML y dibujar las tarjetas dinámicamente mapeando las propiedades de cada objeto.

4. Estructura de la Tabla y Optimización SQL

Se definió la estructura de la tabla `menu` garantizando que cada registro posea una identidad única y campos estrictamente tipados.

SQL

```
CREATE TABLE menu(  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100),  
  descripcion TEXT,  
  imagen VARCHAR(255),  
  precio NUMERIC(10,2)  
);
```

Optimización del Listado General

Durante las pruebas de modificación de productos, el equipo detectó un comportamiento nativo de PostgreSQL: al realizar un `UPDATE`, el motor reubica físicamente la fila modificada al final de la tabla. Para mitigar este impacto visual y evitar que los productos se desordenaran aleatoriamente en la pantalla del administrador, se optimizó la consulta del método `GET` añadiendo un ordenamiento estricto:

SQL

```
SELECT * FROM menu ORDER BY id ASC;
```

Esto asegura que la interfaz mantenga siempre una estructura visual idéntica y predecible para el usuario, independientemente de las ediciones que se realicen.

5. Detalle Fino de las Funcionalidades Desarrolladas

A. Listado Automatizado (Lectura - GET)

Al inicializarse el módulo de administración, se dispara la función asíncrona `cargarProductos()`. Esta realiza un `fetch` hacia la API, recibe el array de productos ordenados y recorre la estructura mediante un ciclo `forEach`, inyectando los nodos de HTML dinámico con las rutas relativas correspondientes para renderizar las imágenes locales (`./recursos/imagenes/`).

B. Alta de Productos (Escritura - POST)

Se capturó el evento `submit` del formulario. Se previene la recarga por defecto de la página (`e.preventDefault()`), se extraen los valores de los inputs y se empaquetan en un objeto JSON. Si el servidor procesa exitosamente la inserción en la base de datos, el formulario se resetea por completo (`e.target.reset()`) y se vuelve a invocar de manera transparente el listado actualizado.

C. Modificación Bidireccional (Actualización - PUT)

La lógica de la modificación requirió un control de estados en el Frontend mediante una variable global (`editandoId`). El proceso se divide en:

- **Fase de Carga:** Al presionar "Modificar", se busca el producto por su ID mediante la API, se inyectan sus valores actuales directamente en los inputs del formulario y se cambia el texto del botón principal a "Guardar Cambios".
- **Fase de Persistencia:** Cuando el usuario envía el formulario con las modificaciones, el código evalúa si `editandoId` contiene un valor. De ser así, redirige la petición utilizando el método `PUT` hacia `/api/productos/:id`. El backend procesa los nuevos datos en la base de datos, se limpia el estado (`editandoId = null`) y el botón retorna a su etiqueta original de "Agregar Producto".

D. Baja Segura de Registros (Eliminación - DELETE)

Para prevenir la pérdida accidental de información crítica del menú, se integró una capa de validación en el cliente mediante una ventana modal de confirmación (`confirm`). La función evalúa la respuesta del administrador: si la acción es cancelada, se interrumpe la ejecución del hilo de JavaScript (`return`), protegiendo la integridad del registro. Si la acción es afirmativa, se despacha la petición `DELETE` hacia el servidor, removiendo físicamente la fila en PostgreSQL y actualizando la interfaz instantáneamente.

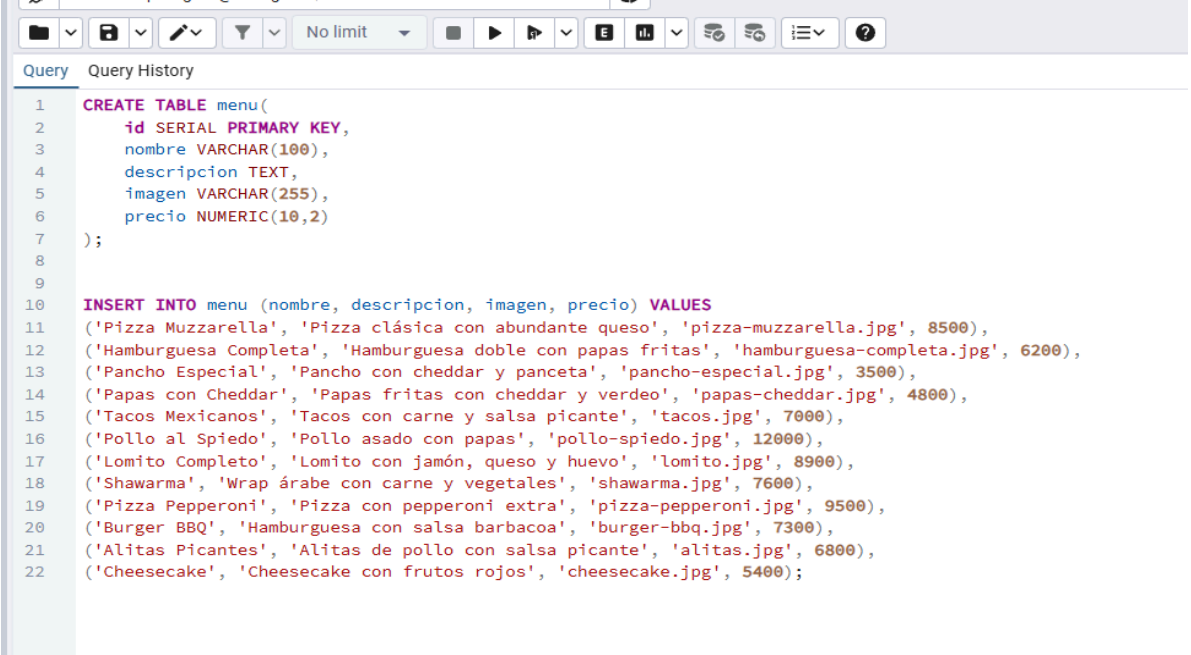
(Nota técnica: Durante el testeo en entornos de producción local, se identificó que las políticas de almacenamiento en caché de ciertos navegadores web ejecutaban versiones obsoletas del script de JavaScript. Esta eventualidad fue sorteada de forma exitosa forzando la limpieza de la memoria caché del navegador e implementando entornos cerrados de pruebas en modo incógnito, garantizando la lectura en caliente del código actualizado).

6. Sección de Evidencias Técnicas (Capturas a Incluir)

(Espacio dispuesto para el pegado de las capturas del software en ejecución)

1.

Captura del Script de Inserción:



```
1 CREATE TABLE menu(  
2     id SERIAL PRIMARY KEY,  
3     nombre VARCHAR(100),  
4     descripcion TEXT,  
5     imagen VARCHAR(255),  
6     precio NUMERIC(10,2)  
7 );  
8  
9  
10 INSERT INTO menu (nombre, descripcion, imagen, precio) VALUES  
11 ('Pizza Muzzarella', 'Pizza clásica con abundante queso', 'pizza-muzzarella.jpg', 8500),  
12 ('Hamburguesa Completa', 'Hamburguesa doble con papas fritas', 'hamburguesa-completa.jpg', 6200),  
13 ('Pancho Especial', 'Pancho con cheddar y panceta', 'pancho-especial.jpg', 3500),  
14 ('Papas con Cheddar', 'Papas fritas con cheddar y verdeo', 'papas-cheddar.jpg', 4800),  
15 ('Tacos Mexicanos', 'Tacos con carne y salsa picante', 'tacos.jpg', 7000),  
16 ('Pollo al Spiedo', 'Pollo asado con papas', 'pollo-spiedo.jpg', 12000),  
17 ('Lomito Completo', 'Lomito con jamón, queso y huevo', 'lomito.jpg', 8900),  
18 ('Shawarma', 'Wrap árabe con carne y vegetales', 'shawarma.jpg', 7600),  
19 ('Pizza Pepperoni', 'Pizza con pepperoni extra', 'pizza-pepperoni.jpg', 9500),  
20 ('Burger BBQ', 'Hamburguesa con salsa barbacoa', 'burger-bbq.jpg', 7300),  
21 ('Alitas Picantes', 'Alitas de pollo con salsa picante', 'alitas.jpg', 6800),  
22 ('Cheesecake', 'Cheesecake con frutos rojos', 'cheesecake.jpg', 5400);
```

2. Captura del Formulario de Edición Activo:

Administración de Productos

Guardar Cambios



Pizza Muzzarella
Pizza clásica con abundante queso
\$8500.00
Modificar Eliminar

3. Captura del Mensaje de Confirmación:

localhost:3000 dice
¿Estás seguro de que querés eliminar este producto?
Aceptar Cancelar

Productos

Guardar Cambios

7. Conclusión

El desarrollo de este Trabajo Práctico permitió consolidar los conocimientos teóricos sobre el backend, las bases de datos relacionales y el comportamiento síncrono/asíncrono de la web. A pesar de la curva de aprendizaje que significó asimilar la lógica de integración de cada componente, el resultado es un sistema administrativo ABM eficiente, seguro y altamente escalable para la rotisería **"Como en casa"**, cumpliendo rigurosamente con los estándares solicitados por la cátedra.

INFORME TÉCNICO: TRABAJO
PRÁCTICO N° 4 – IMPLEMENTACIÓN

DE AUTENTICACIÓN, CONTROL DE ACCESO Y SEGURIDAD

Institución: IES21

Materia: Aplicaciones Web II

Profesor: Andrés Senn

Integrantes: Luján Federico, Giménez Fabrizio

1. Introducción

En esta última etapa del proyecto se incorporaron los mecanismos de autenticación, autorización y seguridad necesarios para transformar la aplicación desarrollada en los trabajos prácticos anteriores en un sistema con acceso restringido para tareas administrativas.

El principal objetivo consistió en implementar un sistema de autenticación basado en JSON Web Tokens (JWT), almacenamiento seguro de credenciales mediante hashing criptográfico y control de acceso utilizando cookies HTTP. De esta forma, únicamente los usuarios autorizados pueden acceder al panel de administración y realizar operaciones de alta, modificación y eliminación de productos.

Asimismo, el proyecto quedó dividido en dos interfaces claramente diferenciadas:

- **Vista para Usuarios (Cliente):** permite consultar el catálogo de productos, agregarlos al carrito de compras, administrar el contenido del carrito y

finalizar una compra simulada mediante una interfaz amigable e intuitiva.

- **Vista para Administradores:** permite iniciar sesión con credenciales válidas y acceder al panel administrativo desde donde se gestionan completamente los productos almacenados en la base de datos mediante operaciones CRUD.

Esta separación de responsabilidades representa una arquitectura mucho más cercana a una aplicación web utilizada en un entorno real.

2. Persistencia de Usuarios en Base de Datos

Como primer paso se incorporó una nueva entidad denominada **usuarios**, almacenada dentro de PostgreSQL.

La tabla fue diseñada específicamente para almacenar la información necesaria para el proceso de autenticación, evitando mantener credenciales en memoria o dentro del código fuente.

La estructura implementada contempla:

- Identificador único.
- Nombre de usuario.
- Contraseña cifrada mediante hashing.

Gracias a esta incorporación, el sistema puede registrar nuevos usuarios y posteriormente validar sus credenciales durante el proceso de inicio de sesión consultando directamente la base de datos.

3. Implementación de Hashing mediante bcrypt

Uno de los aspectos fundamentales del TP fue eliminar el almacenamiento de contraseñas en texto plano.

Para ello se incorporó la biblioteca `bcrypt`, utilizada ampliamente en aplicaciones profesionales debido a que aplica algoritmos criptográficos de una sola vía (One Way Hash).

Durante el registro de un usuario:

- la contraseña ingresada nunca se almacena directamente;
- primero se genera un hash seguro utilizando un Salt automático;
- únicamente dicho hash es persistido en PostgreSQL.

Posteriormente, durante el inicio de sesión, la contraseña ingresada por el usuario es comparada utilizando `bcrypt.compare()`, permitiendo verificar la autenticidad sin necesidad de conocer la contraseña original.

Este procedimiento incrementa significativamente el nivel de seguridad de la aplicación frente a accesos no autorizados o filtraciones de datos.

4. Implementación de Autenticación mediante JWT y Cookies

Para administrar las sesiones de usuario se implementó un sistema basado en JSON Web Token (JWT).

Una vez que el usuario introduce correctamente sus credenciales:

1. El servidor valida el nombre de usuario.
2. Verifica la contraseña utilizando bcrypt.
3. Genera un JWT firmado utilizando una clave secreta.
4. Envía dicho token al navegador mediante una cookie HTTP.

La cookie posee las siguientes características:

- almacenamiento automático por parte del navegador;
- expiración configurada;
- atributo **HttpOnly**, impidiendo el acceso desde JavaScript del lado del cliente.

De esta manera se reduce considerablemente el riesgo de ataques relacionados con el robo de tokens mediante scripts maliciosos.

5. Middleware de Autenticación y Control de Acceso

Se desarrolló un middleware encargado de validar la existencia y autenticidad del JWT recibido en cada petición protegida.

El funcionamiento implementado consiste en:

- leer la cookie enviada por el navegador;
- verificar la firma del token;
- permitir el acceso únicamente cuando el token es válido;
- rechazar cualquier solicitud no autenticada devolviendo un mensaje de autorización denegada.

Este middleware fue aplicado exclusivamente sobre las rutas administrativas que modifican información del sistema.

Como consecuencia:

- cualquier usuario puede consultar el catálogo de productos;
- únicamente un administrador autenticado puede crear, modificar o eliminar productos.

6. Protección del Panel Administrativo

Además de proteger las rutas del backend, también se implementó seguridad desde la interfaz gráfica.

Antes de permitir el acceso a **admin.html**, el sistema realiza automáticamente una verificación de sesión consultando un endpoint específico del servidor.

Si el usuario no posee una sesión válida:

- se bloquea el acceso al panel;
- el navegador redirecciona automáticamente hacia la página de inicio de sesión.

Este mecanismo evita que un usuario pueda acceder simplemente escribiendo manualmente la dirección del panel administrativo.

7. Variables de Entorno

Con el objetivo de proteger información sensible del proyecto se incorporó un archivo `.env`, evitando exponer credenciales dentro del código fuente.

Entre las variables utilizadas se encuentran:

- Usuario de PostgreSQL.
- Contraseña de PostgreSQL.
- Nombre de la Base de Datos.
- Puerto de conexión.
- Clave secreta utilizada para firmar los JWT.

Toda esta información es cargada dinámicamente mediante la biblioteca `dotenv`, facilitando además futuras migraciones del sistema entre distintos entornos de desarrollo y producción.

8. Política CORS

Durante el desarrollo se evaluó la necesidad de incorporar políticas CORS.

Sin embargo, esta configuración no resultó necesaria debido a que tanto el Frontend como el Backend se ejecutan bajo el mismo origen (`localhost:3000`).

Al no existir comunicación entre dominios diferentes, no se presentan restricciones relacionadas con la política Same Origin implementada por los navegadores modernos.

Por este motivo se decidió no incorporar la biblioteca CORS, decisión que se encuentra técnicamente justificada.

9. Mejoras Funcionales Incorporadas

Aprovechando la integración definitiva entre el frontend y el backend, también se realizaron mejoras generales sobre la aplicación.

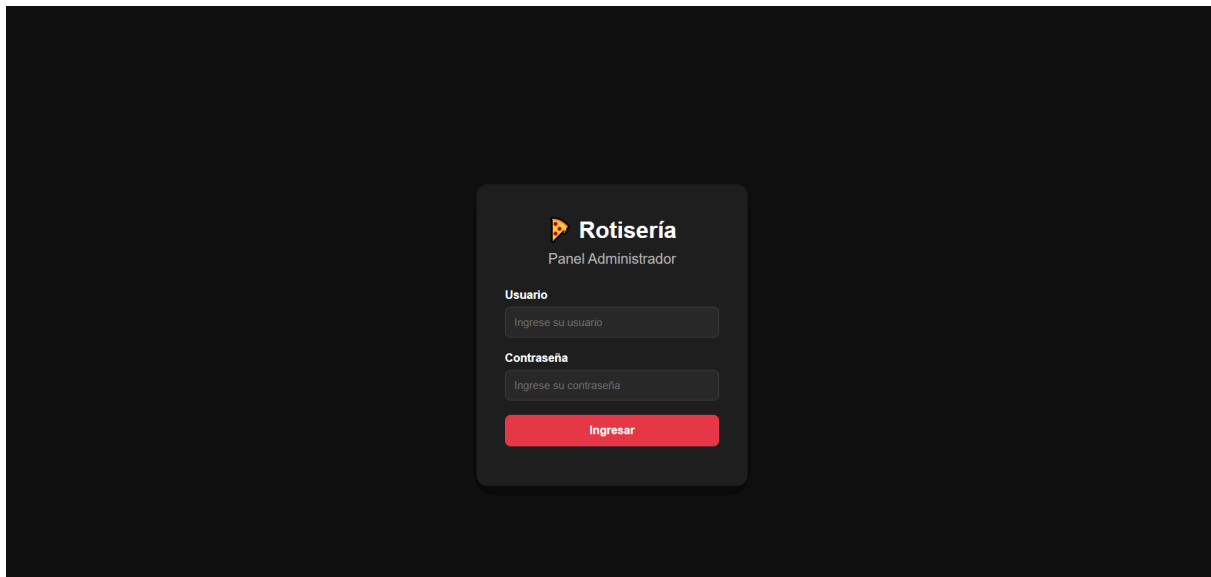
Entre ellas pueden destacarse:

- reemplazo definitivo de MockAPI por la base de datos PostgreSQL como origen de los productos;
- integración completa del catálogo con la API desarrollada durante el TP3;
- creación de una página de inicio de sesión con diseño adaptado a la identidad visual del proyecto;
- protección completa del módulo administrativo;
- implementación de carrito de compras persistente mediante LocalStorage;
- creación de una vista específica para visualizar el carrito;
- posibilidad de eliminar productos individuales o vaciar completamente el carrito;
- incorporación de una pantalla final de **Compra Finalizada**, simulando el cierre del proceso de compra y brindando una experiencia de usuario más completa.

Estas mejoras permitieron transformar el proyecto en una aplicación web mucho más cercana a un escenario real de utilización.

10. Sección de Evidencias Técnicas (Capturas a Incluir)

Captura 1 – Pantalla de Inicio de Sesión



Captura 2 – Catálogo Público



ROTISERÍA COMO EN CASA

"TU COMIDA FAVORITA, LISTA CUANDO LA NECESITAS."



 Pizza Muzzarella Pizza clásica con abundante queso \$8500.00 Agregar al carrito	 Hamburguesa Completa Hamburguesa doble con papas fritas \$6200.00 Agregar al carrito	 Pancho Especial Pancho con cheddar y panceta \$3500.00 Agregar al carrito	 Papas con Cheddar Papas fritas con cheddar y verdeo \$4800.00 Agregar al carrito	 Tacos Mexicanos Tacos con carne y salsa picante \$7000.00 Agregar al carrito	 Pollo al Spiedo Pollo asado con papas \$12000.00 Agregar al carrito
					

Captura 3 – Panel de Administración

Administración de Productos



Pizza Muzzarella

Pizza clásica con abundante queso

\$8500.00



Hamburguesa Completa

Hamburguesa doble con papas fritas

\$6200.00



Pancho Especial

Captura 4 – Carrito de Compras

TU CARRITO

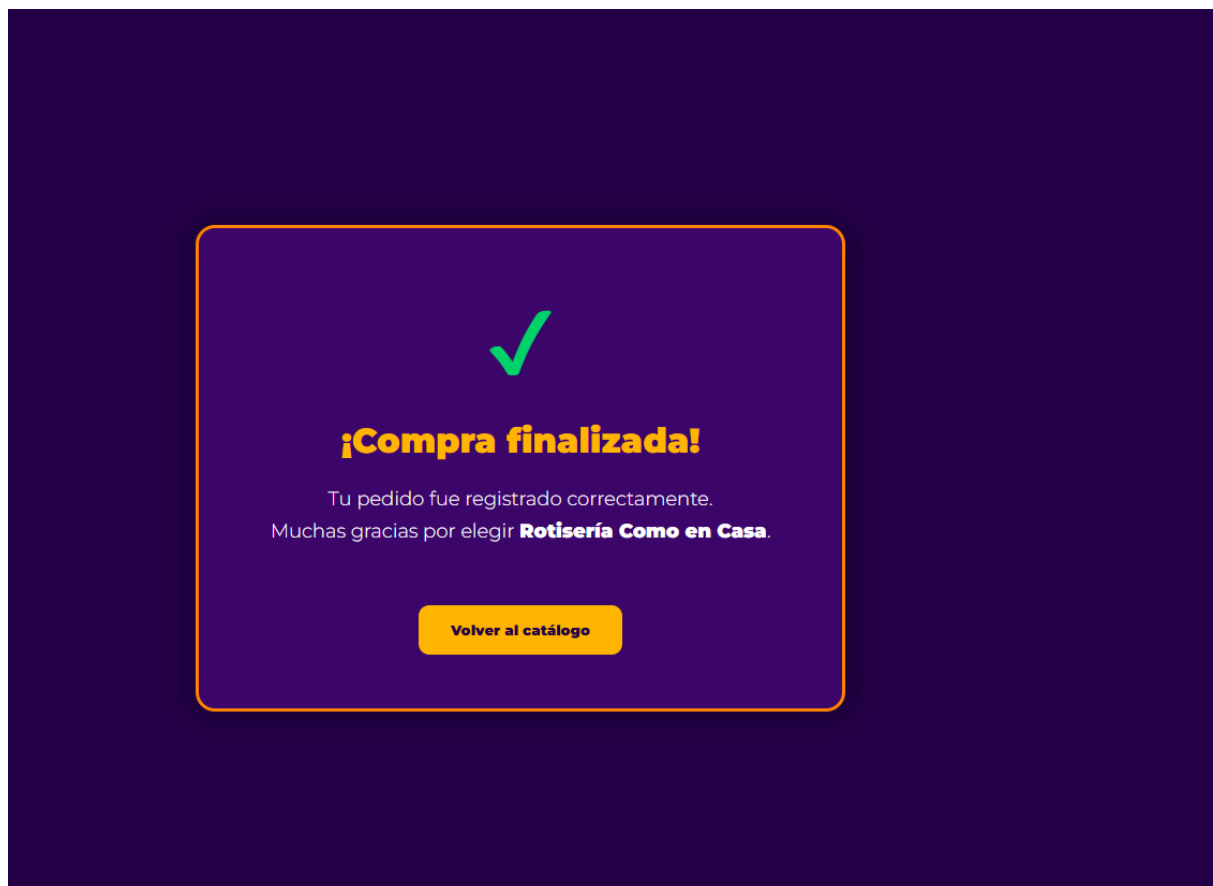
[Volver al catálogo](#)

	Papas con Cheddar	\$4800	Eliminar
	Tacos Mexicanos	\$7000	Eliminar

Total: \$11800

[Vaciar carrito](#)[Finalizar compra](#)

Captura 5 – Compra Finalizada



10. Conclusión

El desarrollo del Trabajo Práctico N.º 4 permitió completar la arquitectura integral del sistema incorporando mecanismos modernos de autenticación y protección de recursos.

La utilización conjunta de PostgreSQL, Express, JWT, Cookies HttpOnly, bcrypt y variables de entorno permitió construir una aplicación considerablemente más segura y organizada, respetando buenas prácticas ampliamente utilizadas en el desarrollo profesional de aplicaciones web.

Como resultado final se obtuvo una plataforma compuesta por dos módulos claramente diferenciados: un **módulo público orientado al cliente**, encargado de la consulta del catálogo y gestión del carrito de compras, y un **módulo privado para administradores**, protegido mediante autenticación, desde el cual es posible gestionar completamente los productos almacenados en la base de datos.

De esta manera se logra consolidar el trabajo desarrollado a lo largo de los cuatro trabajos prácticos, obteniendo una aplicación web funcional, escalable y preparada para futuras ampliaciones, cumpliendo satisfactoriamente con todos los requisitos establecidos por la cátedra.